Inventory Management System (IMS)

System Design Specification

Version: 1.0
System Type: Multi-Tenant Inventory Management Platform
Architecture Style: Modular Monolith with Bounded Context Separation
Design Principles: Domain-Driven Design (DDD), Clean Architecture, API-First Design, Multi-Tenant Isolation, Capability-Based Authorization

---

1. System Overview

1.1 Purpose

The Inventory Management System (IMS) serves as the authoritative source of truth for inventory operations across the E-Commerce Ecosystem.

The platform manages inventory ownership, stock availability, stock reservations, stock allocation, and inventory lifecycle management while exposing secure APIs for both human users and machine integrations.

IMS is intentionally isolated from commerce concerns such as:

- Shopping Carts
- Checkout
- Orders
- Payments
- Product Catalog Management

These responsibilities belong to external systems that consume inventory capabilities through well-defined integration boundaries.

1.2 Core Responsibilities

IMS is responsible for:

Identity & Access

- Account Management
- Client (Machine Identity) Management
- Role Management
- Capability Management
- Authentication
- Authorization

Organization Management

- Organization Lifecycle
- Organization Ownership
- Organization Configuration
- Multi-Tenant Isolation

Inventory Management

- Stock Management
- Stock Availability
- Stock Replenishment
- Batch Tracking
- Stock Allocation

Reservation Management

- Temporary Inventory Reservation
- Reservation Confirmation
- Reservation Release
- Reservation Expiration

Platform Infrastructure

- Auditing
- Logging
- Exception Handling
- API Contracts
- Documentation
- Notifications
- Verification Workflows

---

2. Architectural Principles

2.1 Multi-Tenant Architecture

The platform is designed around organization ownership.

Every inventory resource belongs to a specific organization.

Examples:

- Stock Items
- Stock Batches
- Reservations
- Clients
- Settings

Organizations operate as isolated tenants.

Cross-organization access is prohibited by business authorization policies.

2.2 Actor-Centric Design

The platform is built around a unified Actor abstraction.

Supported actor types:

Human Actors

Accounts

Used for:

- Administration
- Inventory Operations
- Organization Management

Machine Actors

Clients

Used for:

- Service-to-Service Communication
- External Integrations
- Automated Inventory Consumption

The actor model follows Open/Closed principles and allows future actor types without modifying business workflows.

2.3 Capability-Based Authorization

Authorization is based on capabilities rather than hardcoded roles.

Example:

Stock Create
Stock Update
Stock Read
Reservation Confirm
Organization Manage

Roles become collections of capabilities.

This architecture provides fine-grained authorization control while preserving flexibility.

2.4 Bounded Context Separation

IMS intentionally owns:

- Inventory
- Reservations
- Availability
- Allocation

IMS intentionally does not own:

- Products
- Orders
- Checkout
- Payments

These belong to external systems.

---

3. Core Domain Concepts

3.1 Actor

Represents any authenticated identity operating within the platform.

Types:

- Account
- Client

Responsibilities:

- Authentication
- Authorization
- Auditing
- Ownership Attribution

3.2 Organization

Represents a business entity operating inventory.

Responsibilities:

- Inventory Ownership
- Client Ownership
- Settings Management
- Tenant Isolation

3.3 Capability

Represents a permission unit.

Examples:

- stock_create
- stock_update
- reservation_confirm
- organization_manage

3.4 Role

Collection of capabilities assigned to actors.

Responsibilities:

- Permission Grouping
- Access Control Simplification

3.5 Stock Item

Represents an inventory-managed resource.

Responsibilities:

- Availability Tracking
- Reservation Allocation
- Inventory Operations

3.6 Stock Batch

Represents physical inventory quantities.

Tracks:

- Quantity
- Remaining Quantity
- Expiration
- Allocation

Responsibilities:

- FIFO Allocation
- Inventory Consumption

- Reservation Planning

3.7 Reservation

Represents temporary inventory ownership.

Purpose:

Prevent overselling during external business workflows.

Responsibilities:

- Temporary Allocation
- Expiration
- Confirmation
- Release

3.8 Organization Settings

Represents tenant-specific operational configuration.

Examples:

- Reservation Expiration Policy
- Allocation Strategy

---

4. System Components

4.1 Core Module

Provides platform-wide infrastructure.

Contains:

Exception Framework

Categories:

- Business
- Validation
- Policy
- Security
- Technical

API Infrastructure

- Standard Responses
- Pagination
- Metadata
- Error Models

Documentation Infrastructure

- Swagger
- OpenAPI
- Reusable Annotations

Auditing

- Actor Tracking
- Entity Lifecycle Tracking

Logging

- Business Events
- Security Events
- Request Events
- Exception Events

4.2 Identity Module

Responsible for platform identity management.

Contains:

Accounts

Human users.

Roles

RBAC implementation.

Capabilities

Authorization metadata.

Clients

Machine identities.

Verification

Account activation and recovery.

Email

Notification delivery.

4.3 Organization Module

Responsible for tenant management.

Contains:

Organization Management

- Creation
- Updates
- Ownership

Organization Settings

- Reservation Policies
- Allocation Configuration

4.4 Stock Module

Responsible for inventory management.

Contains:

Stock Management

- Create Stock
- Update Stock
- View Stock

Inventory Replenishment

- Restocking

Batch Management

- Batch Tracking
- Allocation Sources

4.5 Reservation Module

Responsible for temporary inventory ownership.

Contains:

Reservation Creation

Temporary stock locking.

Reservation Confirmation

Inventory consumption.

Reservation Release

Stock restoration.

Reservation Expiration

Automatic cleanup.

4.6 Integration Layer

Provides external system connectivity.

Responsibilities:

- Machine Authentication
- Token Management
- API Exposure
- Contract Models

Consumers:

- E-Commerce System
- ERP Systems
- External Partners

---

5. High-Level Data Model

Actor

Attributes:

- ActorCode
- ActorType
- Identity Information

Relationships:

- Assigned Roles
- Audit Ownership

Organization

Attributes:

- OrganizationCode
- Name
- Status

Relationships:

- Accounts
- Clients
- Inventory
- Settings

Role

Attributes:

- RoleCode
- RoleName
- IsDefault

Relationships:

- Capabilities

Capability

Attributes:

- CapabilityCode
- Resource
- Action

Stock Item

Attributes:

- StockCode
- Name
- Availability

Relationships:

- Organization
- Stock Batches

Stock Batch

Attributes:

- BatchCode
- Quantity
- RemainingQuantity
- ExpirationDate

Relationships:

- Stock Item

Reservation

Attributes:

- ReservationCode
- Status
- CreatedAt
- ExpiresAt

Relationships:

- Organization
- Reserved Batches

Organization Settings

Attributes:

- ReservationExpirationMinutes
- AllocationStrategy

Relationships:

- Organization

---

6. End-to-End Workflow

6.1 Stock Creation

1. Authenticated actor submits stock request.
2. Authorization validates capabilities.
3. Organization ownership validated.
4. Stock created.
5. Audit metadata captured.
6. Business event logged.
7. API response returned.

6.2 Inventory Replenishment

1. Organization submits replenishment.
2. Stock batch created.
3. Available quantity increased.
4. Audit recorded.
5. Event logged.

6.3 Reservation Creation

External system calls:

Reserve Stock

Flow:

1. Validate caller.
2. Validate organization ownership.
3. Validate stock existence.
4. Validate available quantity.
5. Allocation engine selects batches.
6. Reservation created.
7. Inventory marked reserved.
8. Expiration timestamp assigned.
9. Response returned.

6.4 Reservation Confirmation

External system calls:

Confirm Reservation

Flow:

1. Validate reservation.
2. Validate reservation state.
3. Consume reserved inventory.
4. Mark reservation confirmed.
5. Update stock quantities.
6. Record audit trail.

6.5 Reservation Release

External system calls:

Release Reservation

Flow:

1. Validate reservation.
2. Release allocated inventory.
3. Restore availability.
4. Mark reservation released.
5. Record event.

6.6 Reservation Expiration

Scheduled process executes:

cleanupExpiredReservations()

Flow:

1. Find expired reservations.
2. Release reserved quantities.
3. Restore inventory.
4. Mark reservations expired.
5. Log operation.

---

7. Reservation Lifecycle State Machine

Reservation States

ACTIVE

Reservation currently holding inventory.

Allowed Transitions:

ACTIVE → CONFIRMED

ACTIVE → RELEASED

ACTIVE → EXPIRED

CONFIRMED

Inventory consumed.

Terminal state.

RELEASED

Inventory returned.

Terminal state.

EXPIRED

Automatically released.

Terminal state.

---

8. Stock Lifecycle

Available

Inventory can be reserved.

↓

Reserved

Inventory temporarily locked.

↓

Confirmed Path

Reserved → Consumed

Inventory permanently deducted.

OR

Release Path

Reserved → Available

Inventory restored.

OR

Expiration Path

Reserved → Available

Inventory automatically restored.

---

9. External System Integrations

E-Commerce System

Consumes:

Reserve Inventory

Creates temporary reservation.

Confirm Reservation

Consumes inventory after payment.

Release Reservation

Restores inventory after payment failure.

ERP Systems

Potential capabilities:

- Inventory Synchronization
- Procurement Integration
- Reporting

Partner Systems

Potential capabilities:

- Inventory Queries
- Reservation Workflows
- Fulfillment Integrations

---

10. Consistency & Transaction Strategy

Internal Consistency

Domain operations execute within transactional boundaries.

Examples:

- Stock Updates
- Reservation Creation
- Reservation Confirmation

Atomicity guaranteed within IMS.

Cross-System Consistency

IMS and E-Commerce remain independently consistent.

Coordination occurs through explicit APIs.

Examples:

Reserve Stock
Confirm Reservation
Release Reservation

No shared database.

No distributed transactions.

Eventual Consistency

Cross-system state synchronization relies on API interactions and scheduled recovery processes.

---

11. Failure Scenarios & Recovery Strategy

Reservation Creation Failure

Cause:

- Insufficient Stock
- Invalid Stock
- Authorization Failure

Result:

- Reservation not created
- Inventory unchanged

Reservation Confirmation Failure

Cause:

- Invalid Reservation
- Already Released
- Already Expired

Result:

- Request rejected
- Inventory unchanged

External System Failure

Cause:

- Network Timeout
- Service Unavailable

Recovery:

- Retry Strategy
- Exception Translation
- Operational Logging

Expiration Job Failure

Cause:

- Infrastructure Outage

Recovery:

- Next scheduled execution processes backlog
- Expired reservations eventually released

Email Delivery Failure

Recovery:

- Retry Policies
- Scheduled Recovery Jobs
- Delivery State Tracking

---

12. Edge Cases

Concurrent Reservation Requests

Multiple systems attempt to reserve same inventory simultaneously.

Handling:

- Transactional validation
- Allocation locking
- Overselling prevention

Reservation Confirmation After Expiration

Scenario:

Reservation already expired.

Handling:

- Confirmation rejected.
- Inventory remains available.

Duplicate Confirmation Request

Scenario:

Reservation already confirmed.

Handling:

- Idempotent validation.
- No inventory changes.

Duplicate Release Request

Scenario:

Reservation already released.

Handling:

- Safe rejection.
- No inventory changes.

Organization Access Violation

Scenario:

Organization attempts access to another organization's inventory.

Handling:

- Authorization failure.
- Access denied.

Client Credential Rotation

Scenario:

Machine secret rotated while integrations active.

Handling:

- Old credentials invalidated.
- New credentials activated.
- Audit trail maintained.

---

13. Security Architecture

Authentication:

- JWT-Based
- Account Authentication
- Client Authentication

Authorization:

- Capability-Based RBAC

Identity Model:

- Actor Abstraction
- Principal Resolution Layer

Security Features:

- Credential Rotation
- Verification Tokens
- Audit Logging
- Security Event Monitoring

---

14. Observability & Audit Architecture

Request Tracing

Every request receives a correlation identifier.

Audit Attribution

All business operations capture:

- Actor Identity
- Timestamp
- Operation Context

Structured Logging

Log Categories:

- Business Events
- Security Events
- Request Events

- Exceptions
- System Operations

Future Evolution

Prepared for:

- OpenTelemetry
- Distributed Tracing
- ELK Stack
- OpenSearch
- Centralized Monitoring

---

15. Future Expansion Roadmap

Planned Inventory Domains:

- Warehouse Management
- Supplier Management
- Purchase Orders
- Inventory Adjustments
- Inventory Transfers
- Inventory Reporting
- Reservation Analytics

Planned Platform Enhancements:

- OAuth2 Client Credentials
- API Key Authentication
- Event-Driven Integrations
- Outbox Pattern
- Distributed Inventory Services
- Multi-Warehouse Allocation Strategies
- Advanced Reservation Policies

Final Architectural Definition

Inventory Management System (IMS) is a multi-tenant, organization-centric inventory platform that serves as the authoritative source of truth for inventory ownership, availability, allocation, and reservation management.
The system combines organization-based tenant isolation, capability-driven authorization, machine-to-machine integrations, reservation lifecycle orchestration, and production-grade operational infrastructure into a scalable architecture designed to support modern commerce ecosystems while remaining independent from order, checkout, payment, and product catalog concerns.